Übersetzung und Analyse von Programmen

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

FORMALE SEMANTIK VON *TRIPLA*

Mathematische Maschine vs. Structural Operation Semantics

$$\delta(x := e; p', b) = (p', b[x/I(b, e)])$$

$$\frac{(b,e)\rightarrow v}{(x{:=}e,b)\rightarrow(\epsilon,b[x/v])}$$

$$\delta(\texttt{while } e \texttt{ do } p_1 \texttt{ od}; p', b) = \begin{cases} (p_1; \texttt{while } e \texttt{ do } p_1 \texttt{ od}; p', b) & \text{if } I(b, e) = true \\ (p', b) & \text{if } I(b, e) = false \end{cases}$$

$$\frac{(b,e)\rightarrow true}{(\texttt{while } e \texttt{ do } p \texttt{ od},b)\rightarrow(p;\texttt{while } e \texttt{ do } p \texttt{ od},b)}$$

$$\frac{(b,e)\rightarrow false}{(\texttt{while } e \texttt{ do } p \texttt{ od},b)\rightarrow(\epsilon,b)}$$

$$\frac{(c,b)\rightarrow(c',b') \qquad (c';p,b')\rightarrow(\epsilon,b'')}{(c;p,b)\rightarrow(\epsilon,b'')}$$

$G_{TRIPLA} = \{$

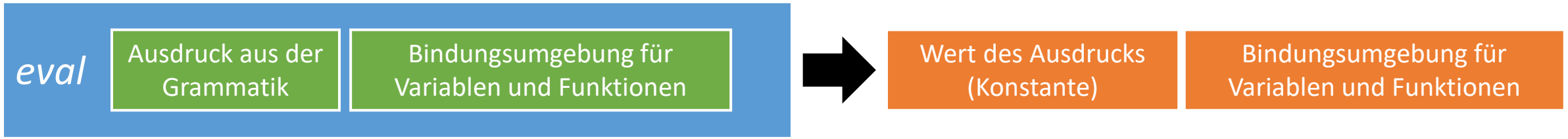
```
E ⟶  let D in E
   |  ID
   |  ID ( A )
   |  E OP E
   |  CONST
   |  ID = E
   |  E ; E
   |  if E then E else E
   |  while E do E
   |  do E while E
A ⟶  E | A , E
D ⟶  ID ( V ) { E } | D D
V ⟶  ID | V , V
ID ⟶ identifier
CONST ⟶ boolean or integer constant
OP ⟶ operators (+,-,*,/,==,!=,<,>)
```

$\}$

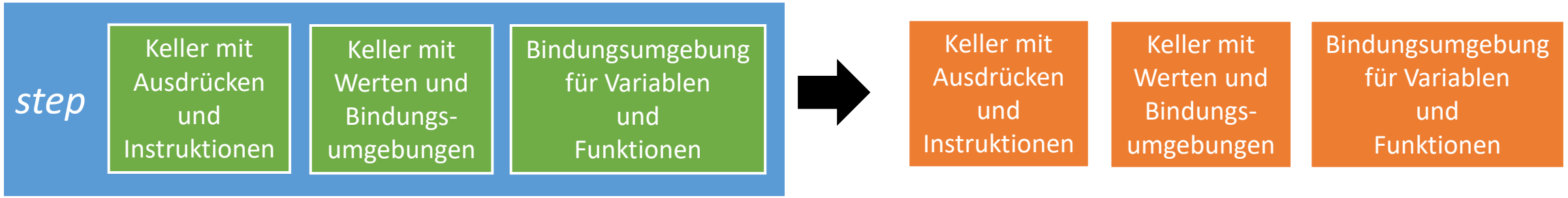
Figure 1: Syntax of TRIPLA

Operationelle Semantiken für TRIPLA

- Big Step Semantics



- Small Step Semantics



Simple Expressions and Assignment

Big Step Semantics

ENV : ID ∪ (ID × N) → N ∪ (ID^* × LG_{TRIPLA}(E))

An environment ρ ∈ ENV binds variables to their values and function names (including their arity) to their definitions.

$$eval(c, \rho) = (c, \rho) \text{ where } c \in \text{CONST}$$

$$eval(x, \rho) = (\rho(x), \rho) \text{ where } x \in \text{ID}$$

$$eval(e_1\ op\ e_2, \rho) = (\overline{op}(v_1, v_2), \rho_2) \text{ where } (v_1, \rho_1 = eval(e_1, \rho) \text{ and } (v_2, \rho_2) = eval(e_2, \rho_1) \text{ and } \overline{op} \text{ is the operation represented by } op$$

$$eval(x{=}e, \rho) = (v, \rho_1[x/v]) \text{ where } (v, \rho_1) = eval(e, \rho)$$

Beispiel:

$$
\begin{aligned}
eval(c, \rho) &= (c, \rho) \text{ where } c \in \mathsf{CONST} \\
eval(x, \rho) &= (\rho(x), \rho) \text{ where } x \in \mathsf{ID} \\
eval(e_1\ op\ e_2, \rho) &= (\overline{op}(v_1, v_2), \rho_2) \\
&\quad \text{where } (v_1, \rho_1 = eval(e_1, \rho) \text{ and } (v_2, \rho_2) = eval(e_2, \rho_1) \\
&\quad \text{and } \overline{op} \text{ is the operation represented by } op \\
eval(x{=}e, \rho) &= (v, \rho_1[x/v]) \\
&\quad \text{where } (v, \rho_1) = eval(e, \rho)
\end{aligned}
$$

eval(x=x−1 , { x↦1 })

=

Control-Flow

Big Step Semantics

$$eval(e_1\,\texttt{;}\,e_2, \rho) = \begin{cases} (v_2, \rho_2) & \text{if } v_2 \neq \emptyset \\ (v_1, \rho_2) & \text{if } v_2 = \emptyset \end{cases}$$
$$\text{where } (v_1, \rho_1) = eval(e_1, \rho) \text{ and } (v_2, \rho_2) = eval(e_2, \rho_1)$$

$$eval(\texttt{if}\, e_1\, \texttt{then}\, e_2\, \texttt{else}\, e_3, \rho) = \begin{cases} eval(e_2, \rho_1) & \text{if } v_1 = \texttt{true} \\ eval(e_3, \rho_1) & \text{if } v_1 = \texttt{false} \end{cases}$$
$$\text{where } (v_1, \rho_1) = eval(e_1, \rho)$$

$$eval(\texttt{while}\, e_1\, \texttt{do}\, e_2, \rho) = \begin{cases} eval(e_2\,\texttt{;while}\, e_1\, \texttt{do}\, e_2, \rho_1) & \text{if } v_1 = \texttt{true} \\ (\emptyset, \rho_1) & \text{if } v_1 = \texttt{false} \end{cases}$$
$$\text{where } (v_1, \rho_1) = eval(e_1, \rho)$$

$$eval(\texttt{do}\, e_1\, \texttt{while}\, e_2, \rho) = eval(e_1\,\texttt{;while}\, e_2\, \texttt{do}\, e_1, \rho)$$

Beispiel: $eval($ **while** x>0 **do** x=x-1 , { x↦1 })

Functions

overwrite restore

Big Step Semantics

$$
\begin{aligned}
eval(&\texttt{let}\, f_1\texttt{(}x_{1,1}\texttt{,}\ldots\texttt{,}x_{1,k_1}\texttt{)\{}e_1\texttt{\}} \\
&\quad\vdots \\
&\quad f_n\texttt{(}x_{n,1}\texttt{,}\ldots\texttt{,}x_{n,k_n}\texttt{)\{}e_n\texttt{\}} \\
&\texttt{in}\, e, \\
&\rho)
\end{aligned}
\quad = \quad
\begin{aligned}
&(v, \rho^*) \\
&\text{where } \rho' = \rho[(f_1,k_1)/((x_{1,1},\ldots,x_{1,k_1}),e_1)] \\
&\qquad\qquad\vdots \\
&\qquad\qquad [(f_n,k_n)/((x_{n,1},\ldots,x_{n,k_n}),e_n)] \\
&\text{and } (v,\rho'') = eval(e,\rho') \\
&\text{and } \rho^* = \rho''[(f_1,k_1)/\rho(f_1,k_1)]\ldots[(f_n,k_n)/\rho(f_n,k_n)]\ _{\iota})]
\end{aligned}
$$

$$
eval(f\texttt{(}e_1\texttt{,}\ldots\texttt{,}e_n\texttt{)},\rho) \quad = \quad
\begin{aligned}
&(v,\rho^*) \\
&\text{where } \rho(f,n) = ((x_1,\ldots,x_n),e) \\
&\text{and} \quad (v_1,\rho_1) = eval(e_1,\rho), \\
&\qquad\quad (v_2,\rho_2) = eval(e_2,\rho_1), \\
&\qquad\quad\vdots \\
&\qquad\quad (v_n,\rho_n) = eval(e_n,\rho_{n-1}) \\
&\text{and } \rho' = \rho_n[x_1/v_1]\ldots[x_n/v_n] \\
&\text{and } (v,\rho'') = eval(e,\rho') \\
&\text{and } \rho^* = \rho''[x_1/\rho_n(x_1)]\ldots[x_n/\rho_n(x_n)]
\end{aligned}
$$

Beispiel:

eval(**let** `inc(x){ x=x-1}` **in** `inc(x+5),{x ⊢ 1})=`

Simple Expressions and Assignment

Small Step Semantics

Next, we define a TRIPLA machine $M = (K, K^f, step)$ where $K = CODE \times STACK \times ENV$ is the set of configurations and $K^f = \{(stop :: r, s, \rho) \in K\}$ the set of final configurations. We use a tuple to store intermediate results. We use the :: operator to add an entry at the first position of a tuple: $a :: (a_1, \ldots, a_n) = (a, a_1, \ldots, a_n)$. Since we only add and remove elements at the beginning of this tuple, we call this tuple a stack.

$$
\begin{aligned}
step(c :: r, s, \rho) &= (r, c :: s, \rho) \text{ where } c \in \mathsf{CONST} \\
step(x :: r, s, \rho) &= (r, \rho(x) :: s, \rho) \text{ where } x \in \mathsf{ID} \\
step(e_1\ op\ e_2 :: r, s, \rho) &= (e_1 :: e_2 :: op :: r, s, \rho) \\
step(op :: r, v_2 :: v_1 :: s, \rho) &= (r, \overline{op}(v_1, v_2) :: s, \rho) \\
step(x{=}e :: r, s, \rho) &= (e :: \texttt{store}(x) :: r, s, \rho) \text{ where } x \in \mathsf{ID} \\
step(\texttt{store}(x) :: r, v :: s, \rho) &= (r, v :: s, \rho[x/v]) \text{ where } x \in \mathsf{ID}
\end{aligned}
$$

Small Step Semantics

Control-Flow

$$
\begin{aligned}
step(e_1\,\texttt{;}\,e_2 :: r, s, \rho) &= (e_1 :: e_2 :: \texttt{checkempty} :: r, s, \rho) \\
step(\texttt{checkempty} :: r, v_2 :: v_1 :: s, \rho) &= \begin{cases} (r, v_1 :: s, \rho) & \text{if } v_2 = \emptyset \\ (r, v_2 :: s, \rho) & \text{otherwise} \end{cases} \\
step(\texttt{if}\,e_1\,\texttt{then}\,e_2\,\texttt{else}\,e_3 :: r, s, \rho) &= (e_1 :: \texttt{iftrue(}e_2\texttt{,}e_3\texttt{)} :: r, s, \rho) \\
step(\texttt{iftrue(}e_1\texttt{,}e_2\texttt{)} :: r, \texttt{true} :: s, \rho) &= (e_1 :: r, s, \rho) \\
step(\texttt{iftrue(}e_1\texttt{,}e_2\texttt{)} :: r, \texttt{false} :: s, \rho) &= (e_2 :: r, s, \rho) \\
step(\texttt{while}\,e_1\,\texttt{do}\,e_2 :: r, s, \rho) &= (e_1 :: \texttt{iftrue(}e_2\texttt{;while}\,e_1\,\texttt{do}\,e_2\texttt{,}\ \texttt{empty))} :: r, s, \rho) \\
step(\texttt{empty} :: r, s, \rho) &= (r, \emptyset :: s, \rho) \\
step(\texttt{do}\,e_1\,\texttt{while}\,e_2 :: r, s, \rho) &= (e_1\,\texttt{;while}\,e_2\,\texttt{do}\,e_1 :: r, s, \rho)
\end{aligned}
$$

Functions

overwrite restore

Small Step Semantics

$$
\begin{array}{rcl}
step(\texttt{let}\, f_1(x_{1,1},\ldots,x_{1,k_1})\{e_1\}, & = & (e :: \texttt{restore\_funs}((f_1,k_1),\ldots,(f_n,k_n)) :: r,\\
\quad\vdots & & \rho :: s,\\
\quad f_n(x_{n,1},\ldots,x_{n,k_n})\{e_n\} & & \rho_1)\\
\quad \texttt{in}\, e :: r & & \text{where } \rho_1 = \rho\,[(f_1,k_1)/((x_{1,1},\ldots,x_{1,k_1}),e_1)]\\
\quad s, & & \qquad\vdots\\
\quad \rho) & & \qquad [(f_n,k_n)/((x_{n,1},\ldots,x_{n,k_n}),e_n)]\\
step(\texttt{restore\_funs}((f_1,k_1),\ldots,(f_n,k_n)) :: r, & = & (r, v :: s, \rho[(f_1,k_1)/\rho'(f_1,k_1)]\ldots[(f_n,k_n)/\rho'(f_n,k_n)])\\
\quad v :: \rho' :: s, & & \\
\quad \rho) & & \\
step(f(e_1,\ldots,e_n) :: r, s, \rho) & = & (e_1 :: \ldots :: e_n :: \texttt{invoke}(f,n) :: r, s, \rho)\\
step(\texttt{invoke}(f,n) :: r, v_n :: \ldots :: v_1 :: s, \rho) & = & (e :: \texttt{restore\_vars}(x_1,\ldots,x_n) :: r,\\
 & & \rho :: s,\\
 & & \rho_n[x_1/v_1]\ldots[x_n/v_n])\\
 & & \text{where } \rho(f,n) = ((x_1,\ldots,x_n),e)\\
step(\texttt{restore\_vars}(x_1,\ldots,x_n) :: r, v :: \rho' :: s, \rho) & = & (r, v :: s, \rho[x_1/\rho'(x_1)]\ldots[x_n/\rho'(x_n)])
\end{array}
$$